



Instituto Politécnico Nacional

Escuela Superior de Computo



Materia: Aplicaciones para Comunicaciones en Red

Profesor: Tanibet Pérez De los Santos Mondragón

Integrantes: Gil Valentín Oswaldo

Grupo: 6CV1

Fecha: 27/03/2026

Actividad: Ejercicio 2.5 Demultiplex selectores

Ejercicio 2.5 : Servidor TCP tipo “sistema de monitoreo de sensores” con selectores

Desarrollar un servidor TCP que:

1. Acepte múltiples clientes simultáneamente (cada cliente = un sensor).
2. Use modo no bloqueante.
3. Utilice un selector (selectors o equivalente).

```
def accept_client(sock_a, mask):  
    """Función callback para aceptar nuevos sensores."""  
    sock_conn, addr = sock_a.accept()  
    print(f"\n[+] Nuevo sensor aceptado desde {addr}")  
  
    sock_conn.setblocking(False)  
  
    sel.register(sock_conn, selectors.EVENT_READ, read_sensor_data)
```

Aceptamos a cada cliente y le asignamos su propio socket, lo identificamos por sus datos de dirección IP y Puerto.

4. Procesa datos enviados periódicamente por los clientes.

```
try:
    data = sock_c.recv(1024)

    if data:
        # Procesamiento de los datos enviados por el sensor
        mensaje = data.decode('utf-8').strip()
        addr = sock_c.getpeername()
        print(f"[Sensor {addr[0]}:{addr[1]}] Datos recibidos: {mensaje}")

        #Enviar una confirmación de vuelta al sensor
        respuesta = f"Servidor ACK: '{mensaje}' procesado.\n".encode('utf-8')
        sock_c.sendall(respuesta)

    else:
        addr = sock_c.getpeername()
        print(f"[-] Sensor {addr} desconectado.")
        sel.unregister(sock_c)
        sock_c.close()

except ConnectionResetError:
    print(f"[!] Un sensor se desconectó de forma abrupta.")
    sel.unregister(sock_c)
    sock_c.close()
```

Requisitos

Cada cliente (sensor) envía datos en formato texto:

sensor_id,valor

Ejemplo:

temp01,23.5

```
PS C:\Users\Hideki GM\Desktop\Ejercicio 2.5 Demultiplex selectores> python Cliente.py
Iniciando Sensor #783...
Conectado al servidor central. Iniciando envío periódico...

[Sensor 783] Enviando: Temp: 23.14°C
[Sensor 783] Respuesta del servidor: Servidor ACK: 'Temp: 23.14°C' procesado.
```

El servidor debe:

- Recibir datos de múltiples sensores.
- Mostrar en consola:

Sensor temp01 reporta: 23.5

```
oswaldo-gv@oswaldo-gv-VirtualBox: ~/Descargas/Selectores
oswaldo-gv@oswaldo-gv-VirtualBox:~/Descargas/Selectores$ python3 '/home/oswaldo-gv/D
gv/Descargas/Selectores/SelectServer.py'
Servidor de Sensores (Selectors) escuchando en el puerto 5050...
[*] Total inicial de sensores activos: 0

[+] EVENTO DE CONEXIÓN: Nuevo sensor conectado desde ('192.168.56.106', 60283)
[*] Total de sensores activos: 1
[Sensor 60283] Datos: Temp: 24.34°C
[Sensor 60283] Datos: Temp: 23.89°C

[+] EVENTO DE CONEXIÓN: Nuevo sensor conectado desde ('192.168.56.106', 60288)
[*] Total de sensores activos: 2
[Sensor 60288] Datos: Temp: 27.2°C

[+] EVENTO DE CONEXIÓN: Nuevo sensor conectado desde ('192.168.56.106', 60291)
[*] Total de sensores activos: 3
[Sensor 60291] Datos: Temp: 26.55°C
[Sensor 60283] Datos: Temp: 27.06°C
[Sensor 60288] Datos: Temp: 24.51°C
[Sensor 60291] Datos: Temp: 28.1°C
```

- Manejar desconexiones correctamente.

```
oswaldo-gv@oswaldo-gv-VirtualBox: ~/Descargas/Selectores
[Sensor 60283] Datos: Temp: 26.96°C
[Sensor 60288] Datos: Temp: 21.26°C
[Sensor 60291] Datos: Temp: 24.77°C
[Sensor 60283] Datos: Temp: 22.05°C
[Sensor 60288] Datos: Temp: 25.99°C

[-] EVENTO DE DESCONEXIÓN: El sensor ('192.168.56.106', 60283) cerró la conexión
correctamente.
[*] Total de sensores activos: 2
[Sensor 60291] Datos: Temp: 25.04°C
[Sensor 60288] Datos: Temp: 21.47°C
[Sensor 60291] Datos: Temp: 29.48°C
[Sensor 60288] Datos: Temp: 25.24°C

[-] EVENTO DE DESCONEXIÓN: El sensor ('192.168.56.106', 60288) cerró la conexión
correctamente.
[*] Total de sensores activos: 1
[Sensor 60291] Datos: Temp: 28.62°C
[Sensor 60291] Datos: Temp: 25.0°C

[-] EVENTO DE DESCONEXIÓN: El sensor ('192.168.56.106', 60291) cerró la conexión
correctamente.
[*] Total de sensores activos: 0
```

Detección de sensores inactivos

Establecer un valor de timeout de inactividad para detectar sensores que dejan de enviar datos. Una vez identificado se debe borrar de la lista de sensores activos.

Responde:

1. ¿Cómo el sistema operativo identifica a qué conexión pertenece cada dato recibido?

El Sistema Operativo identifica la conexión mediante la Tupla de 5. Al comparar esta combinación única con su tabla interna de sockets activos, logra dirigir cada flujo de datos a la aplicación correcta de forma independiente

2. ¿Cómo el selector ayuda a manejar múltiples sensores?

El selector ayuda a manejar múltiples sensores permitiendo la concurrencia en un solo hilo. Logra esto mediante la notificación de eventos, evitando que el procesador trabaje innecesariamente.

3. ¿Por qué este enfoque escala mejor que usar un hilo por sensor?

El enfoque de selectores escala mejor porque optimiza los recursos más valiosos del servidor. RAM y CPU, evitando el desperdicio que genera crear miles de hilos "dormidos" que no están haciendo nada la mayor parte del tiempo.

4. ¿Qué problemas aparecerían con miles de sensores reales?

El cuello de botella ya no es solo el código de Python, sino los límites del sistema operativo, la capacidad de la red y la seguridad del canal donde se implementan los selectores.

Registro de eventos

Cada vez que se registre un evento de conexión o desconexión mostrar en pantalla:

- Número de sensores activos

```
[.] EVENTO DE DESCONEXIÓN: El sensor ('192.168.56.106', 60283) cerró la conexión correctamente.  
[*] Total de sensores activos: 2
```